PBT205 Task 1 Report: RabbitMQ Chat Application

Oct 21, 2026 · @Seyla Song

1. Overview

The prototype is a command-line chat app (`chat.py`). Any number of users can join a named room and exchange messages in real time through a RabbitMQ broker. It is a single Python file of 151 lines that uses the `pika` client library.

In scope

- Start-up from the command line: `python chat.py <username> <port> [room]` .
- Broadcast: every message reaches every other user in the same room.
- Concurrent sending and receiving in one terminal.
- Clear error messages for bad arguments and for a broker that cannot be reached.
- Clean exit on Ctrl+C or Ctrl+D, which removes the user's queue from the broker.

Extension built: multiple chat rooms. The room name is the routing key, so users in different rooms never see each other's messages.

Out of scope: message history, login or authentication, private messages, a graphical interface, and clients on machines other than the broker's host.

`send.py` and `receive.py` are the RabbitMQ "Hello World" tutorial scripts, used only to check that the broker worked. They are not part of the Task 1 solution.

2. Requirements and constraints

The brief sets the middleware (RabbitMQ). Each design choice below follows from a requirement, and each one adds a constraint that the next choice has to respect.

Requirement	What it forces	Resulting constraint
Middleware is RabbitMQ, run in Docker	Clients speak AMQP 0-9-1 through `pika`	The broker must be running before any client starts

Requirement	What it forces	Resulting constraint
Every user in a room sees every message	Each user needs their own queue, not one shared queue	One queue per connected user
Rooms are kept apart	A `direct` exchange with the room name as routing key	Room names must match exactly (case-sensitive)
Start-up takes username and port	Arguments are read and checked in `parse_args()`	Host is fixed to `localhost`; only the port is configurable
Users can type while messages arrive	Receive on a background thread, read input on the main thread	`pika` is not thread-safe, so sends go through `add_callback_threadsafe`
Messages carry sender and time	A JSON body: `username`, `message`, `timestamp`	Every client must agree on these field names

Protocol. AMQP 0-9-1 over TCP on port 5672 (RabbitMQ, n.d.-a). Port 15672 serves the management UI, which was used to inspect exchanges, queues and bindings during testing.

Serialisation (JSON). JSON was chosen because it is human-readable and a standard format (Bray, 2017). It ships with Python's `json` module (Python Software Foundation, n.d.), and any later client in another language can read it. It also lets the message grow new fields without breaking older clients. The cost is a few dozen bytes of overhead per message, which does not matter at chat volumes.

Integration. The broker is the only shared component. Clients never connect to each other, so any program that publishes the same JSON to the `chat_rooms` exchange can join a room.

Boundary conditions

- Wrong number of arguments, or a port that is not a number: print usage and exit with code 1.
- Broker not reachable at start-up: print the Docker command to start it and exit with code 1.
- Broker lost during a session: tell the user and exit.
- Room left out: prompt for one, defaulting to `general`.

- Empty lines: not sent.
- Late joiners: receive only messages sent after they join. There is no history by design.

3. Design

Every client publishes to one `direct` exchange, `chat_rooms`, using the room name as the routing key. Each client also owns a private queue bound to its room, so the broker copies each message to everyone in that room and to no one else.

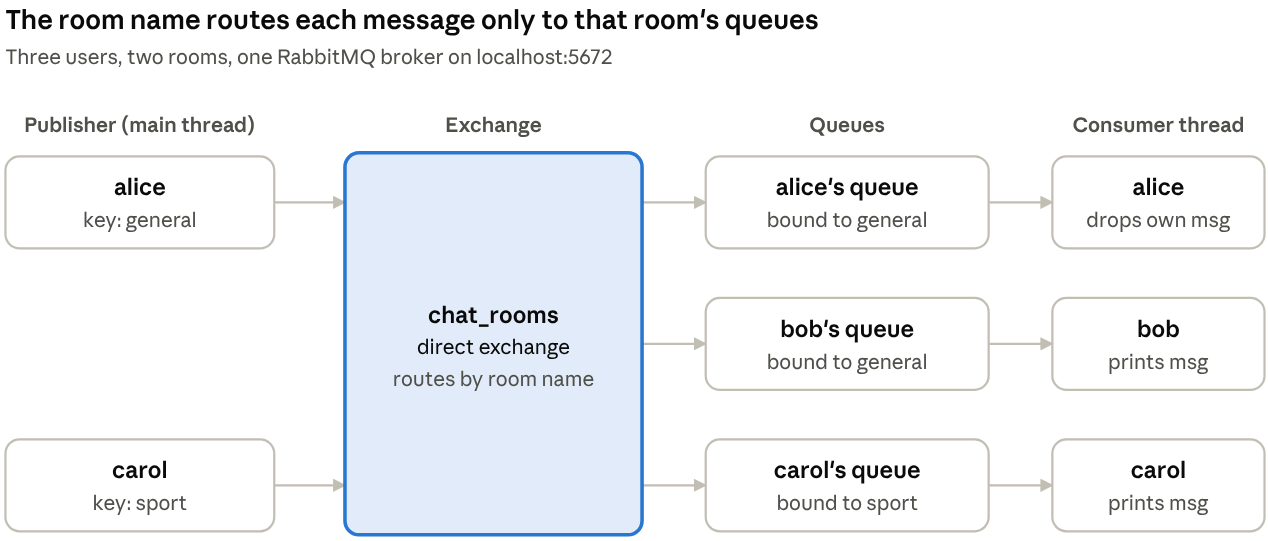


Figure 1. Architecture: one direct exchange, one exclusive queue per user

Alice's message to `general` reaches her own queue and Bob's, but not Carol's. Alice's consumer drops the copy of her own message, because she has already seen what she typed.

Why a direct exchange and one queue per user

Option	What happens	Verdict
One shared queue that all users consume	RabbitMQ round-robins: each message goes to only one user	Rejected: chat needs broadcast, not work sharing
`fanout` exchange	Every message goes to every queue, whatever the room	Rejected: works for one room only, breaks room isolation

Option	What happens	Verdict
`topic` exchange	Routing by pattern, such as `sport.*`	Possible, but wildcards are not needed for flat room names
`direct` exchange, room name as routing key, queue per user	Broadcast inside the room, nothing leaks to other rooms	**Chosen**

Each queue is declared with `queue=""` (the broker picks a unique name) and `exclusive=True` (only this connection can use it, and the broker deletes it when the connection closes). This means no leftover queues pile up on the broker when users leave.

Message contract

Every message body is a UTF-8 JSON object with three fields:

```
{
  "username": "alice",
  "message": "hello from alice",
  "timestamp": "2026-10-21T20:41:07.512334"
}
```

- `username`: the sender, shown before the message and used to skip your own echo.
- `message`: the text as typed, with leading and trailing spaces removed.
- `timestamp`: the sender's local time in ISO 8601. Receivers show it as `HH:MM`.

The room is not in the body. It travels as the routing key, so the broker can route on it without reading the message.

Start-up arguments

Argument	Required	Meaning	Check
`username`	Yes	Name shown to other users	None (any string)
`port`	Yes	RabbitMQ TCP port, normally 5672	Must parse as an integer
`room`	No	Room to join	Prompted for if left out; default `general`

Threading: typing and receiving at once

`input()` blocks the thread that calls it, and so does `channel.start_consuming()`. One thread cannot do both, so the app uses two:

- **Main thread:** reads lines from the keyboard.
- **Consumer thread** (daemon): runs `start_consuming()` and prints each incoming message.

`pika`'s `BlockingConnection` is not thread-safe. So the main thread never calls `basic_publish` itself. It hands the send to the consumer thread with `connection.add_callback_threadsafe()`, the one method `pika` documents as safe to call from another thread.

Terminal interface

The app has no GUI; the interface is the terminal. A `> ` prompt shows where to type. Incoming messages print as `[HH:MM] bob: hi alice`. Each one starts with `\r` so it overwrites the bare prompt instead of appearing after it, and then the prompt is printed again. Ctrl+C or Ctrl+D prints `Leaving chat...` and closes the connection.

4. Implementation

The app is one file, `chat.py`, written in Python with `pika` 1.4.4. It was tested on Python 3.14.8 against RabbitMQ 3.13.7 (Docker image `rabbitmq:3-management`) on macOS.

How to run it

1. Start the broker: `docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3-management`
2. Create a virtual environment and install `pika`: `python3 -m venv venv`, `source venv/bin/activate`, then `pip install pika==1.4.4`.
3. Run one client per terminal: `python chat.py alice 5672 general`

Key code

Setting up the exchange, the private queue and the room binding (`chat.py:78-86`):

```
channel.exchange_declare(exchange=EXCHANGE_NAME, exchange_type="direct")
result = channel.queue_declare(queue="", exclusive=True)
queue_name = result.method.queue
channel.queue_bind(exchange=EXCHANGE_NAME, queue=queue_name,
routing_key=room)
```

Handing a send from the input thread to the connection thread (`chat.py:137`):

```
connection.add_callback_threadsafe(lambda t=text: publish(t))
```

The `t=text` default argument captures the current line. Without it, a lambda that runs later could read a newer value of `text`.

Skipping your own echo and printing over the prompt (`chat.py:94-101`):

```
if data["username"] == username:
    return
print(f"\r[{time}] {data['username']}: {data['message']}")
print("> ", end="", flush=True)
```

Clean shutdown, so the exclusive queue is deleted (`chat.py:144-147`):

```
if connection.is_open:
    connection.add_callback_threadsafe(channel.stop_consuming)
    consumer_thread.join(timeout=5)
    connection.close()
```

Attribution

- The connect, declare, publish and consume pattern follows the official RabbitMQ Python tutorials 1, 3 and 4 (RabbitMQ, n.d.-b, n.d.-c, n.d.-d). `send.py` and `receive.py` are adapted from tutorial 1 and use a durable quorum queue.
- The cross-thread send uses `add_callback_threadsafe` as described in the `pika` documentation (Pika, n.d.).
- The broker runs from the official Docker image (Docker Hub, n.d.).

5. Testing and validation

15 of 17 tests passed against a live broker on 21 October 2026. The two failures are real bugs, listed under "Bugs found" below. All core requirements passed: broadcast to 3+ users, room isolation, late joiners and broker failure.

Method

- **Integration tests:** a Python harness started real `chat.py` processes as separate clients. It typed into each one's standard input and checked each one's output. The broker was RabbitMQ 3.13.7 in Docker on `localhost:5672`.
- **Broker checks:** the management HTTP API (`/api/exchanges/%2F/chat_rooms/bindings/source`) was used to count queue bindings before and after clients left.
- **Broker failure:** a second, throwaway broker was started on port 5673 and stopped with `docker stop` while a client was connected. This left the main broker untouched.
- **Argument and start-up errors:** run by hand from the shell, with exit codes recorded.

Test results

ID	Test	Input / steps	Expected	Actual	Result
T1	Broadcast to 3 users	alice, bob, dave in `general`; alice and bob each send	Each message shown to the other two	Shown to the other two	Pass
T2	Room isolation	carol in `sport` sends; alice sends in `general`	Neither room sees the other's message	carol saw only her own prompt	Pass
T3	No self-echo	alice sends "hello from alice"	Not printed back to alice	Not printed	Pass

ID	Test	Input / steps	Expected	Actual	Result
T4	Late joiner, no history	eve joins `general` after T1	Earlier messages not shown	Not shown	Pass
T5	Late joiner, new messages	alice sends "welcome eve"	eve receives it	Received	Pass
T6	Unicode	"héllo 👋 你好"	Shown unchanged	Shown unchanged	Pass
T7	Empty line	User presses Enter on a blank line	Nothing sent	Nothing sent	Pass
T8	Long message	5,000 characters	Delivered in full	Delivered in full	Pass
T9	Ordering	alice sends seq0 to seq19 quickly	bob sees them in order	In order	Pass
T10	Clean exit removes queue	3 clients send SIGINT (Ctrl+C)	Bindings drop by 3	6 → 3; each printed `Leaving chat...`	Pass
T11	Wrong number of arguments	`python chat.py`	Usage message, exit 1	Usage message, exit 1	Pass
T12	Port not a number	`python chat.py seyla abc general`	Error, exit 1	`Error: port must be a number, got 'abc'`, exit 1	Pass
T13	Broker unreachable at start	Port 5999 (nothing listening)	Clear error, exit 1	Error plus the Docker start command, exit 1	Pass
T14	Broker down mid-session	`docker stop` on the test broker	User told, app exits	`Lost connection to RabbitMQ. Press Enter to exit.`, exited 5 s later	Pass
T15	Room left out	`python chat.py seyla 5672`, then Enter	Joins `general`	`Joined room 'general' as seyla.`	Pass
T16	Two users with the same name	Second "alice" joins and sends	First alice sees it	First alice does not see it	**Fail (B3)**
T17	Malformed message	A non-JSON body (`not json`) published to `general`	Ignored, chat carries on	Every client in `general` crashed with `JSONDecodeError`	**Fail (B2)**

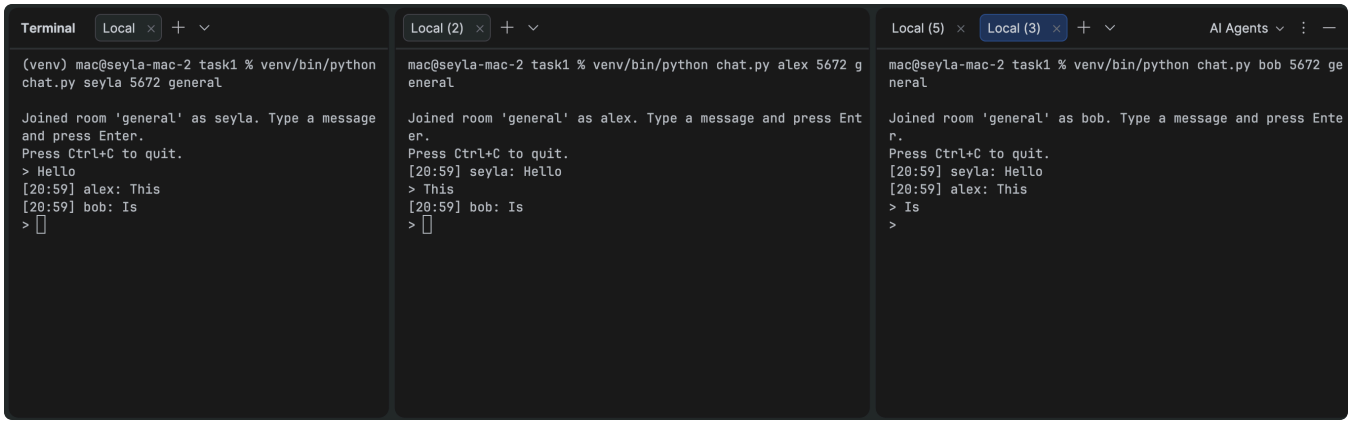


Figure 1: Screenshot of the chat application during testing

Bugs found

ID	Bug	Cause	Impact	Fix
B2	One bad message crashes the whole room	`json.loads(body)` in `on_message` (`chat.py:90`) has no error handling. The exception ends `start_consuming()`, and `consume()` only catches `AMQPError`	Any program on the broker can stop every user in a room with a single message. A missing field (`KeyError`) or bad timestamp (`ValueError`) does the same	Wrap the parsing in `try/except (ValueError, KeyError)` and skip bad messages
B3	Two users with the same name cannot see each other	The self-echo filter compares usernames (`chat.py:94`), not senders	Messages are silently hidden	Add a per-client id (for example `uuid4()`) to each message and filter on that, or reject duplicate names

How the results change the solution

- **B2** shows that a client should never trust what comes off the broker. Checking each message before using it turns a room-wide crash into one skipped message.

- **B3** shows that a display name is not an identity. A per-client id fixes this without asking users to do anything.
- T14 confirmed that the decision to catch `AMQPError` in the consumer thread works: users get a clear message instead of a stack trace.
- T10 confirmed the choice of `exclusive=True` queues: nothing is left on the broker after users leave.

6. Hardware and software constraints

The prototype depends on one local broker and keeps nothing on disk. That is fine for a classroom demo, but it limits availability, reach and scale.

Constraint	Detail	Effect	Mitigation
Docker dependency	The broker runs in the `rabbitmq:3-management` container	No Docker (or a port clash on 5672) means no chat	A native RabbitMQ install, or a hosted broker
Single broker, single point of failure	Every client connects to one broker	If it stops, all chat stops (T14)	A RabbitMQ cluster with quorum queues
No persistence	Non-durable exchange, exclusive auto-deleted queues, `auto_ack=True`, no history	Late joiners see nothing earlier (T4). Messages in flight are lost if the broker or a client fails	Durable queues, persistent messages and manual acks, plus a history store
Localhost only	Host is fixed to `localhost` in `connect()`; RabbitMQ's `guest` user is also limited to localhost by default	All users must be on the broker's machine	A `--host` argument and a dedicated broker user
No authentication	Anyone who can reach the broker can read or post in any room, under any name	Impersonation, plus the crash in B2	Per-user broker accounts and permissions

Constraint	Detail	Effect	Mitigation
Scaling	One queue and one TCP connection per user. Each message is copied once per user in the room	Broker memory and fan-out work grow with users × message rate	Fine for tens of users; larger loads need clustering and load testing, which was not done
Python and pika	Needs Python 3 and `pika` 1.4.x	The client must have Python installed	Package the client, for example with PyInstaller

The hardware used for testing was a single Mac laptop running both the broker and all clients. No performance limits were reached at the volumes tested (up to 6 clients at once and 20 messages in a burst).

7. Conclusion and lessons learned

The prototype meets the Task 1 brief. Users chat in real time through RabbitMQ, rooms are kept apart, and the app handles bad arguments and a lost broker cleanly. Testing found two bugs, each with a small fix.

- The exchange type decides the behaviour. A shared queue spreads messages across users, while a queue per user broadcasts them. Picking `direct` with room names as keys gave rooms almost for free.
- Thread safety matters even in a small app. `pika` connections must stay on one thread, so `add_callback_threadsafe` was essential.
- Validate every message received. One unchecked `json.loads` let a single bad message stop a whole room (B2).
- Testing with real processes against a real broker found bugs that reading the code had missed (B2, B3).

8. References and appendix

References

Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) data interchange format* (RFC 8259). Internet Engineering Task Force. https://doi.org/10.17487/RFC8259

Docker Hub. (n.d.). *rabbitmq: Official image*. Retrieved October 21, 2026, from https://hub.docker.com/_/rabbitmq

Pika. (n.d.). *BlockingConnection*. Pika documentation. Retrieved October 21, 2026, from https://pika.readthedocs.io/en/stable/modules/adapters/blocking.html

Python Software Foundation. (n.d.). *json: JSON encoder and decoder*. Python documentation. Retrieved October 21, 2026, from https://docs.python.org/3/library/json.html

RabbitMQ. (n.d.-a). *AMQP 0-9-1 model explained*. Retrieved October 21, 2026, from https://www.rabbitmq.com/tutorials/amqp-concepts

RabbitMQ. (n.d.-b). *RabbitMQ tutorial: "Hello World!" (Python)*. Retrieved October 21, 2026, from https://www.rabbitmq.com/tutorials/tutorial-one-python

RabbitMQ. (n.d.-c). *RabbitMQ tutorial: Publish/subscribe (Python)*. Retrieved October 21, 2026, from https://www.rabbitmq.com/tutorials/tutorial-three-python

RabbitMQ. (n.d.-d). *RabbitMQ tutorial: Routing (Python)*. Retrieved October 21, 2026, from https://www.rabbitmq.com/tutorials/tutorial-four-python

Appendix A: Test environment

Item	Version
OS	macOS
Python	3.14.8
pika	1.4.4
RabbitMQ	3.13.7, Docker image `rabbitmq:3-management`